

Adaptive Retrieval Planning and Agentic Re-retrieval for Enterprise Knowledge Management

Won Young Shin

Department of Information Systems, Hanyang University

pingu090@hanyang.ac.kr

June 2026

Abstract

Enterprise knowledge retrieval poses distinctive challenges that are not addressed by general-purpose information retrieval research: a single document corpus routinely contains API technical references, human resources policy manuals, security compliance guides, and project meeting notes, each demanding different retrieval signals for effective access. We present two complementary contributions to this problem. First, we construct the Synthetic Enterprise Knowledge Dataset (SEKD), a fully annotated 120-document, 1,206-chunk corpus with 405 queries spanning six reasoning types and eight difficulty factors, together with ground-truth chunk-level answer annotations and oracle retrieval strategy labels. Using SEKD, we conduct the first controlled, systematic comparison of five retrieval systems on enterprise knowledge queries: BM25 sparse retrieval, dense retrieval with BGE-small-en-v1.5, Hybrid Reciprocal Rank Fusion (RRF), a seven-rule adaptive planner, and zero-shot GPT-5 strategy planning. Our results establish that Hybrid RRF dominates fixed-strategy baselines on all primary metrics (Recall@10= 0.745, MRR= 0.530, nDCG@10= 0.566), that a seven-rule deterministic planner achieves the highest Recall@10 (0.755) among all systems at zero operational cost, and that GPT-5 provides ranking quality improvements (+2.0% MRR, +1.2% nDCG@10) at \$6.18 per 380 queries. Critically, all evaluated systems fail systematically on exception clause queries (Recall@10 $\leq$ 0.412) and temporal reasoning queries (Recall@10 $\leq$ 0.200). Second, motivated by these persistent failures, we introduce the V2 Agentic RAG pipeline: a five-agent LangGraph system that wraps the V1 retrieval infrastructure in an iterative validation–re-retrieval loop. A Validation Agent scores retrieved chunks against per-type relevance thresholds, diagnoses failure modes on rejection, and provides strategy recommendations for corrective re-retrieval. We design four controlled ablation experiments (E1–E4) to isolate the contributions of the re-retrieval loop, validation scoring quality, and query classification accuracy. The results establish which agentic components add measurable value beyond static planning and identify the failure modes that remain intractable after iterative correction.

Keywords: enterprise retrieval, hybrid RRF, adaptive retrieval planning, agentic RAG, re-retrieval loop, LangGraph, BM25, dense passage retrieval, enterprise knowledge management.

1 Introduction

Modern enterprises generate and maintain large quantities of heterogeneous structured and semi-structured knowledge. A typical mid-sized technology organization maintains separate document repositories for API technical references, human resources policy manuals, system architecture documents, project meeting notes, security compliance guides, and operational policy documents such as travel expense policies. As these corpora grow in volume and variety, the challenge of locating relevant information efficiently becomes a first-order organizational problem. Employees searching across hundreds of policy documents,

engineers querying system design archives, and automated compliance checking systems all depend on retrieval systems that surface the right information reliably.

Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for knowledge-intensive natural language processing (Lewis et al., 2020). In a RAG system, an upstream retrieval component retrieves a set of candidate documents or passages from a corpus, and a downstream language model conditions its output on those retrieved passages. The retrieval step is therefore the critical bottleneck: if relevant documents are not retrieved, no amount of downstream generation quality can compensate. The quality of retrieval is measured along two primary dimensions — *coverage* (does the system find all relevant documents?) and *ranking quality* (does it place the most relevant documents highest?).

Enterprise corpora present three structural challenges that distinguish them from the web search and open-domain question answering settings that dominate IR research. First, *query heterogeneity*: enterprise queries span a wide range of reasoning types, from simple single-hop factual lookups (“What is the hotel reimbursement limit?”) to complex multi-hop reasoning across multiple documents (“Which API endpoints use the same authentication method as the data export service?”), temporal ordering queries (“Which policy version was current during Q3 2024?”), policy exception clause extraction (“Under what conditions is an employee exempt from pre-approval?”), and multi-attribute aggregation tasks (“List all security sections mandating two-factor authentication”). Different reasoning types benefit from fundamentally different retrieval signals: temporal queries require exact date-token matching that dense embeddings cannot capture; exception clause queries require semantic identification of conditional language that BM25 vocabulary matching cannot detect; multi-hop queries benefit from broad document coverage.

Second, *corpus heterogeneity*: the same retrieval method cannot be optimal across an API reference (identifier-rich, structure-heavy) and an HR policy (semantics-heavy, exception-dense). A dense encoder fine-tuned for semantic similarity matching may excel on paraphrased policy queries while failing on queries that require matching exact version numbers or endpoint paths. Conversely, a BM25 index excels at exact identifier matching but has no mechanism for semantic paraphrase recognition.

Third, *single-pass rigidity*: all standard retrieval architectures make the strategy selection decision once per query and have no mechanism to recover if initial results are insufficient. When a query yields a low-relevance result set, there is no feedback loop to diagnose why retrieval failed or to attempt an alternative strategy. This rigidity is particularly costly for the exception and temporal query types, where the mismatch between query semantics and retrieval signal is structural rather than parametric.

Adaptive retrieval planning addresses the second challenge by dynamically routing each query to the retrieval backend best suited to its characteristics. A planner examines query metadata (reasoning type, named entities, difficulty factors) and selects among available retrieval strategies. However, adaptive planning alone cannot address the third challenge: if the planner’s initial strategy selection is incorrect, or if the initial retrieval is insufficient regardless of strategy, no correction mechanism exists.

This paper investigates both adaptive planning and agentic re-retrieval as solutions to enterprise knowledge retrieval. We organize our investigation around thirteen research questions. RQ1–RQ3 establish the retrieval baseline landscape: does Hybrid RRF outperform single-method baselines; which method is best by document category; how does performance

vary by reasoning type? RQ4–RQ7 evaluate adaptive planning: can rule-based planning outperform fixed-strategy retrieval; which LLM prompt strategy is optimal; does GPT-5 outperform the rule planner; what are LLM planning’s failure modes? RQ8–RQ13 evaluate the V2 agentic extension: does the pipeline outperform V1 planners; what is the contribution of each agentic component; does re-retrieval address exception and temporal failure; what failure modes remain?

The experimental program proceeds in two phases. *V1 evaluation* (Sections 3–6) characterizes the performance of all five retrieval systems on SEKD, establishes the performance ceiling for single-pass retrieval under ideal conditions, and identifies the failure modes that no static planning mechanism can address. *V2 evaluation* (Section 5) introduces the agentic pipeline architecture and four controlled ablation experiments (E1–E4), together with the empirical results that reveal a critical threshold calibration issue in the Validation Agent.

Our contributions are as follows. **(1) Dataset:** We construct and release SEKD, a controlled enterprise corpus with 405 annotated queries supporting systematic evaluation of retrieval methods, planners, and agentic systems. **(2) V1 baseline study:** We report the first systematic comparison of five retrieval systems on enterprise knowledge queries, establishing empirically grounded performance benchmarks across reasoning types and document categories. **(3) V2 agentic system:** We implement a five-agent LangGraph pipeline with iterative validation–re-retrieval, targeting the exception and temporal failure modes identified in V1 while preserving V1 planning components as fallback. **(4) Ablation study:** We isolate the contributions of re-retrieval, adaptive validation scoring, and query classification accuracy through controlled experiments. **(5) Practical recommendations:** We provide quantitative, actionable design guidelines for enterprise RAG practitioners based on the complete experimental program.

2 Related Work

2.1 Sparse and Dense Retrieval

The BM25 (Robertson and Zaragoza, 2009; Robertson et al., 1995) family of sparse retrieval models score documents by term frequency and inverse document frequency with saturation and normalization parameters. The Okapi BM25 variant (Equation 1) applies a sub-linear TF saturation function with parameter k_1 that prevents high-frequency terms from dominating the score, and a document length normalization term with parameter b that adjusts for the advantage of longer documents.

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f_{t,d} \cdot (k_1 + 1)}{f_{t,d} + k_1 \cdot (1 - b + b \cdot |d|/\text{avgdl})} \quad (1)$$

where $f_{t,d}$ is the term frequency of term t in document d , $|d|$ is document length, avgdl is the corpus average document length, and $\text{IDF}(t) = \log \frac{N - n_t + 0.5}{n_t + 0.5}$ for corpus size N and document frequency n_t . In our implementation, $k_1=1.5$ and $b=0.75$ follow standard enterprise retrieval configurations. Dense Passage Retrieval (DPR) (Karpukhin et al., 2020) introduced bi-encoder dense retrieval, training question and passage encoders separately and retrieving via approximate nearest neighbor search. Xiong et al. (2021) scaled dense retrieval using FAISS-based approximate search; Gao and Callan (2021) improved dense encoders through pre-fine-tuning with dense-retrieval-specific pre-training objectives. BAAI/bge-small-en-v1.5,

used in our study, is a compact (384-dimensional) bi-encoder achieving competitive BEIR benchmark performance (Thakur et al., 2021) at low computational cost. For enterprise contexts, Oguz et al. (2022) showed that domain-matched pre-training of dense models substantially improves retrieval quality on specialized corpora.

2.2 Hybrid Retrieval and Fusion

Cormack et al. (2009) introduced Reciprocal Rank Fusion (RRF) as a parameter-free fusion method combining multiple ranked lists by accumulating reciprocal rank scores. RRF’s main advantage is that it requires no training, is robust to differences in score magnitude across retrieval systems, and tolerates the absence of a document from one ranked list. Luan et al. (2021) demonstrated that sparse-dense hybrid retrieval consistently outperforms either unimodal approach, particularly on queries with complementary lexical and semantic requirements. Ma et al. (2022) provided an empirical analysis of fusion sensitivity to fusion parameters, confirming that RRF is competitive with learned fusion weights for diverse query distributions. In this paper, we confirm both complementarity and RRF fusion advantages on the SEKD enterprise query distribution.

2.3 Adaptive Retrieval Planning and Query Routing

Query routing — selecting a retrieval method or model based on query characteristics — has been studied in the IR literature under labels including query classification (Salton, 1971), selective retrieval, and query-adaptive fusion. Nogueira and Cho (2019) showed that query type moderates retrieval performance and that query features can predict which retrieval method will be most effective. Zhang et al. (2023) proposed a learned query router for dense versus sparse retrieval selection. LLM-based query analysis for retrieval planning was explored by Ma et al. (2023), who showed that LLM-generated query expansions improve retrieval quality for underspecified queries. Shi et al. (2023) demonstrated that irrelevant context injected into LLM prompts can substantially degrade reasoning quality, motivating careful prompt engineering for planning tasks. Our GPT-5 evaluation provides the first assessment of frontier LLM-based retrieval planning, finding that explicit structured metadata in the prompt is more important than model scale for strategy classification accuracy.

2.4 Enterprise Information Retrieval

Enterprise information retrieval has received comparatively less attention in the academic literature than web search or open-domain question answering, despite its practical importance. The enterprise setting differs from web IR in several structural ways: document corpora are finite and curated (typically thousands to millions rather than billions of documents), documents are heterogeneous by design (policy documents, technical manuals, and meeting notes coexist in the same corpus), and queries are often high-stakes (compliance verification, legal research, technical specification lookup). Early enterprise IR research focused on structured query models for document management systems (Baeza-Yates and Ribeiro-Neto, 1999); more recent work has examined enterprise search in the context of intranet retrieval, email search, and knowledge base question answering.

The TREC Enterprise Track (Voorhees and Harman, 2005) established benchmarks for expert search and known-item search in enterprise corpora, finding that BM25 with

query expansion performs competitively with more complex models. BEIR (Thakur et al., 2021) provided a cross-domain zero-shot retrieval benchmark including enterprise-adjacent domains (scientific papers, clinical documents, legal text), finding that dense retrieval models generalize better than BM25 to semantically distant query-document pairs. Our work contributes a corpus specifically designed to capture the structural properties of enterprise knowledge corpora — including the category mix, reasoning type distribution, and difficulty factor profile — while enabling controlled experiments not possible on real-world proprietary corpora.

2.5 Agentic and Iterative RAG

The ReAct framework (Yao et al., 2023) introduced the reasoning-action interleaving paradigm for LLM agents, enabling iterative tool use guided by internal chain-of-thought reasoning. In the ReAct paradigm, the LLM alternates between generating reasoning traces (“Thought”) and executing actions (“Act: Search[X]”) with observed outcomes (“Observation”) feeding back into the next reasoning step. Crucially, ReAct’s retrieval decisions are made by the same LLM that generates the final answer, creating tight coupling between generation quality and retrieval coverage. Our V2 design decomposes these responsibilities: a dedicated Query Analysis Agent determines what to retrieve, a dedicated Planning Agent determines how to retrieve it, and a dedicated Validation Agent assesses retrieval quality separately from the generation process.

FLARE (Jiang et al., 2023) dynamically decides when to retrieve additional context during generation, triggering retrieval when the LLM’s generation confidence (measured by token probability) falls below a threshold. FLARE’s triggering mechanism is implicit and generation-driven: retrieval occurs as a side effect of low-confidence generation, not as a result of explicit evidence quality assessment. Self-RAG (Asai et al., 2024) goes further by training a model to predict special critique tokens indicating whether retrieval is needed (“[Retrieve]”), whether retrieved passages are relevant (“[IsREL]”), and whether the generated response is supported by the retrieved evidence (“[IsSUP]”). Self-RAG’s critique tokens require task-specific fine-tuning, limiting applicability to domains where supervised training data is available.

MuSiQue (Trivedi et al., 2022) introduced compositional multi-hop question answering that requires evidence from multiple documents retrieved in a specific order. MuSiQue queries are constructed by composing single-hop questions where the answer to each sub-question is required to formulate the next sub-question, motivating sequential retrieval architectures with slot-filling between retrieval steps. Our V2 RA’s sequential mode implements exactly this pattern for multi_hop reasoning type queries.

Our V2 system differs from Self-RAG and FLARE in a key design dimension: rather than embedding retrieval decisions within LLM generation, we implement retrieval validation as a standalone scoring function that operates before generation, enabling clean separation between retrieval quality assessment and answer synthesis. This separation means that V2 retrieval quality can be evaluated independently of generation quality, using the same Recall@K metrics as V1, enabling direct comparison across the experimental program. LangGraph (LangChain, Inc., 2024) provides the stateful directed graph framework for the V2 agent implementation, supporting conditional edges (the re-retrieval back-edge), shared typed state across agent nodes, and clean agent-level modularity that allows individual

components to be replaced or ablated independently.

3 Dataset: The Synthetic Enterprise Knowledge Dataset (SEKD)

3.1 Corpus Construction

The SEKD corpus comprises 120 documents across six enterprise document categories: (1) *API Documentation* — technical reference documents describing endpoint specifications, authentication schemes, rate limits, and integration guides; (2) *HR Policies* — employment policy documents covering leave entitlements, performance review processes, compensation guidelines, and disciplinary procedures; (3) *Project Meeting Notes* — structured meeting minutes containing decisions, action items, project status updates, and dependency tracking; (4) *Security Policies* — compliance and security governance documents covering authentication requirements, data handling mandates, access control specifications, and incident response procedures; (5) *System Design Documents* — architectural specifications describing system components, data flows, integration patterns, and deployment configurations; (6) *Travel Policies* — operational policy documents specifying reimbursement limits, approval workflows, booking procedures, and expense categories.

Each document was generated using a Jinja2 template pipeline with randomized entity values: project names, employee IDs, API endpoint paths, policy section codes, monetary thresholds, and date ranges. Randomization was seeded at a fixed value for reproducibility while producing sufficient variety to prevent trivial keyword matching. Entity values are drawn from domain-appropriate distributions: API endpoint paths follow REST naming conventions (e.g., `/v2/payments/authorize`, `/api/v1/users/{id}/roles`); HR policy codes follow alphanumeric section numbering conventions (e.g., `HR-3.2.1`); meeting note dates are drawn from a realistic 18-month project timeline with consistent project and stakeholder names across documents. Cross-document consistency is deliberately included for multi-hop query types: a system design document and its corresponding API reference share entity names so that multi-hop queries can require synthesizing information across both.

The resulting corpus contains 1,206 section-aware chunks. Chunking follows a three-tier strategy: (1) split on section headers with preserved section metadata (document ID, category, section path, section number); (2) recursively split oversized sections on paragraph boundaries with 50-token overlap to prevent context-boundary information loss; (3) merge undersized terminal sections (fewer than 50 tokens) with their preceding sibling section to prevent trivially short chunks from appearing in retrieval results. Mean chunk length is 248 tokens; the interquartile range is 156–327 tokens, reflecting the diversity of section lengths across document categories. All chunks retain document ID, section path, category, and position metadata as structured fields, enabling category-filtered retrieval and section-level provenance in the Answer Agent.

3.2 Query Set Construction and Annotation

The 405-query set was constructed via a three-phase process. In the *generation phase*, query templates were instantiated for each document, with queries designed to cover all reasoning types and difficulty factors. In the *annotation phase*, a verbatim evidence extraction procedure identified the minimal set of SEKD chunks whose text contains the complete, non-decomposable factual basis for each answer, producing a ground-truth chunk-ID set

per query. In the *quality phase*, queries with ambiguous evidence, queries requiring world knowledge unavailable in SEKD, and structural duplicates were removed. The final set contains 380 answerable queries (used in all evaluations) and 25 unanswerable queries.

Reasoning type distribution: `single_hop` (220, 57.9%), `multi_hop` (53, 13.9%), `aggregation` (35, 9.2%), `comparison` (33, 8.7%), `exception` (34, 8.9%), `temporal` (5, 1.3%). The `single_hop` dominance reflects the realistic distribution of enterprise queries: the majority of lookup queries (“What is the expense limit for a domestic flight?”, “Which team is responsible for the authentication microservice?”) are direct single-hop retrievals requiring one document chunk. The underrepresentation of temporal queries (1.3%) reflects the challenge of constructing well-formed temporal queries over a synthetic corpus with limited historical document variety.

Oracle retrieval strategy labels were assigned using a heuristic mapping grounded in the retrieval signal analysis: `temporal`→`bm25` (exact date-token matching is necessary), `exception`→`dense` (semantic identification of conditional clause language is necessary), `multi_hop`→`hybrid` (cross-document coverage maximizes recall for dependency chains), `aggregation`→`hybrid` (multi-document pooling requires broad coverage), `comparison`→`hybrid` (cross-document semantic comparison requires both lexical and semantic signals), `single_hop`→`strategy` determined by query content and category (BM25 for identifier-heavy API and Travel queries; Dense for semantics-heavy HR and Security queries; Hybrid as default). The oracle label distribution is: Hybrid (249, 65.5%), Dense (69, 18.2%), BM25 (62, 16.3%). The Hybrid majority reflects the conservative, broad-coverage bias in the oracle assignment for multi-hop and aggregation types.

3.3 Difficulty Factors and Annotation Protocol

Eight difficulty factors annotate query complexity and guide the analysis of system performance. *temporal_dependency*: the query requires temporal ordering or date-sequence reasoning. *exception_clause*: the answer is an exception to a general policy rule, typically introduced by “unless,” “except when,” “provided that,” or similar conditional language. *multi_document_dependency*: the complete answer requires synthesizing evidence from two or more distinct documents. *table_dependency*: the answer is encoded in a structured table within a document rather than in prose. *document_version_conflict*: multiple document versions within the corpus contain the same policy with conflicting values, and the query requires identifying the correct version. *negation*: the answer requires interpreting negated conditions. *pronoun_resolution*: the answer depends on resolving ambiguous pronouns across sentences or sections. *implicit_reasoning*: the answer requires inference that is not explicitly stated in any single chunk.

Annotation of difficulty factors was performed alongside ground-truth chunk annotation. For each query, the annotator identified the relevant chunk set by verbatim evidence extraction: locating the specific sentences in the corpus whose concatenation constitutes the minimal non-decomposable factual basis for the answer. A query was flagged unanswerable if (a) no chunk in the corpus contains sufficient evidence for the answer, (b) the answer requires world knowledge not present in SEKD, or (c) the evidence span is ambiguous across multiple conflicting chunks. The final answerable set (380 queries) and unanswerable set (25 queries) provide both a primary evaluation benchmark and an out-of-distribution generalization test.

Difficulty factor distribution in the answerable set: `temporal_dependency` (5, 1.3%), `exception_clause` (34, 8.9%), `multi_document_dependency` (53, 13.9%), `table_dependency` (41, 10.8%), `document_version_conflict` (18, 4.7%), `negation` (29, 7.6%), `pronoun_resolution` (22, 5.8%), `implicit_reasoning` (67, 17.6%). Queries may carry multiple difficulty factors simultaneously (e.g., a temporal query may also require pronoun resolution). The distribution confirms that the dataset is substantially more complex than a simple factual lookup benchmark.

3.4 Evaluation Protocol

Retrieval evaluation treats each answerable query as a recall-precision task over chunk IDs. A chunk is *retrieved* if it appears in the top- k results returned by the retrieval system. A chunk is *relevant* if it is a member of the ground-truth chunk-ID set for that query. Standard metrics Recall@K, MRR, nDCG@K, and Hit@K are computed at $k \in \{1, 3, 5, 10\}$. All systems are evaluated on the same 380 answerable queries with the same corpus, enabling direct metric comparison.

We report the primary evaluation results at $k = 10$, which reflects a realistic retrieval budget for a RAG pipeline that conditions an LLM generation on retrieved passages. At $k = 10$, a single-answer factual query may retrieve multiple relevant chunks (e.g., a policy table and an explanatory paragraph) while a multi-hop query may capture supporting evidence for all required reasoning steps. Recall@1 is additionally reported as a proxy for precision-at-top-1, which is particularly important for applications where only the top result is used (e.g., a slot-filling pipeline that reads only the first retrieved chunk). MRR and nDCG@10 combine coverage and ranking quality, with nDCG@10 placing greater weight on the position of relevant results within the top-10 window.

4 V1 Retrieval Systems

4.1 Retrieval Backends

BM25. We implement BM25 using the standard Okapi scoring function with $k_1=1.5$, $b=0.75$, $\epsilon=0.25$ (floor for negative IDF terms). The corpus is tokenized with a standard English tokenizer with stopword removal. BM25 benefits from exact token matching and is effective for queries containing precise identifiers (policy codes, version numbers, API endpoint paths, monetary thresholds) but is limited for queries requiring semantic paraphrase recognition or exception clause interpretation.

Dense Retrieval. We encode documents and queries using BAAI/bge-small-en-v1.5 (384 dimensions). Documents are encoded offline and stored in a FAISS IVF index with 8 clusters. At query time, the query is encoded independently; retrieval uses exact inner product search over L2-normalized vectors, equivalent to cosine similarity. Dense retrieval benefits from semantic matching but encodes temporal ordering signals poorly: date tokens that carry precise sequential meaning receive similar embedding representations, producing zero retrieval recall on temporal queries. This failure is fundamental to the representation: without explicit temporal position encodings or fine-tuning on temporal contrast pairs, dense embeddings cannot distinguish “Q3 2024” from “Q4 2024” as ordered temporal references.

Hybrid RRF. We fuse BM25 and Dense ranked lists using Reciprocal Rank Fusion (Cormack et al., 2009) with $k=10$. The fused score for chunk c is $\text{RRF}(c) =$

$\sum_{i \in \{\text{bm25}, \text{dense}\}} (k + r_i(c))^{-1}$, where $r_i(c)$ is the rank of chunk c in list i (or ∞ if not present). RRF is parameter-free (no training), handles missing chunks gracefully, and is robust to differences in score magnitude across retrieval systems.

4.2 Implementation Details

The BM25 index is constructed using a standard inverted index with IDF weights computed over the 1,206-chunk corpus. Chunk tokens are lowercased and stripped of punctuation; stopwords are removed using a 127-term English stopwords list. No query expansion, pseudo-relevance feedback, or document length normalization beyond the standard Okapi BM25 formula is applied. At retrieval time, BM25 returns up to $k=10$ candidates ranked by descending score.

The Dense FAISS index stores L2-normalized BGE-small-en-v1.5 embeddings (384 dimensions) in an IVF-flat index with 8 Voronoi cells, trained on the corpus chunk population. At query time, the query string is tokenized and encoded using the same BGE-small-en-v1.5 model (with the `Represent this sentence:` prefix, following model documentation), and the top- k nearest neighbors are retrieved using exact inner product search over L2-normalized vectors. The IVF index probes all 8 cells at query time to ensure exact recall within the indexed set.

Hybrid RRF fusion is performed in memory: both BM25 and Dense ranked lists of 10 results are merged into a dictionary keyed by chunk ID, and each chunk’s RRF score is computed as $\text{RRF}(c) = \sum_{i \in \{\text{bm25}, \text{dense}\}} (k + r_i(c))^{-1}$ with $k = 10$. Chunks present in only one list receive $r_j = \infty$ for the absent list, contributing zero to that term. The merged list is re-ranked by descending RRF score and truncated to $k = 10$.

4.3 Rule-Based Adaptive Planner

The rule planner evaluates seven deterministic rules in priority order (Table 1). Rules R01–R04 target reasoning types where the optimal strategy is unambiguous: temporal queries contain date tokens requiring BM25 exact matching; exception queries contain conditional modifiers requiring Dense semantic understanding; multi-hop and aggregation queries require Hybrid broad coverage. Rules R05–R06 target difficulty factors associated with specific document structures. Rule R07 is the unconditional Hybrid default.

Table 1: Rule Planner Decision Logic: Seven Rules in Priority Order

Rule	Condition	Strategy	SSA
R01	reasoning_type = temporal	BM25	100%
R02	reasoning_type = exception	Dense	100%
R03	reasoning_type = multi_hop	Hybrid	100%
R04	reasoning_type = aggregation	Hybrid	100%
R05	difficulty = document_version_conflict	BM25	N/A
R06	difficulty = table_dependency	Dense	N/A
R07	(default for all unmatched)	Hybrid	83.9%
Overall SSA			78.4%

The planner’s primary weakness is BM25 under-prediction: it routes only 5 of 62 oracle-BM25 single-hop queries to BM25 (8.1% recall on oracle-BM25 single-hop). Single-hop

queries in API documentation and Travel Policies categories that benefit from exact identifier matching are defaulted to Hybrid by R07. Additional rules targeting keyword-dense query patterns could push SSA above 80% but would require per-category heuristics.

4.4 Per-Category Retrieval Performance

Table 2 reports Recall@10 broken down by document category for the three retrieval backends and the Rule Planner. Category-level analysis reveals that document structure determines retrieval difficulty more than any other single factor.

Table 2: Recall@10 by Document Category. Best per-category in bold.

Category	n	BM25	Dense	Hybrid	Rule	GPT-5
API Documentation	68	0.734	0.751	0.755	0.773	0.763
HR Policies	54	0.649	0.701	0.702	0.712	0.718
Meeting Notes	100	0.772	0.784	0.793	0.787	0.783
Security Policies	64	0.696	0.721	0.729	0.741	0.747
System Design	45	0.703	0.756	0.752	0.748	0.740
Travel Policies	49	0.693	0.718	0.727	0.744	0.741
All	380	0.713	0.740	0.745	0.755	0.750

API Documentation queries show the largest absolute variance across systems (+3.9 pp from BM25 to Rule Planner). API documents are rich in exact identifiers — endpoint paths, version strings, HTTP method names, status codes — making BM25 a strong competitor and making the rule planner’s targeted BM25 routing (R01, R05) particularly effective. Dense embeddings capture semantic similarity between query intent and document content but lose precision on exact identifiers, explaining why Dense underperforms the Rule Planner by 2.2 pp for this category. GPT-5’s performance for API Documentation (0.763) reflects category confusion failures: GPT-5 over-predicts Dense for API queries containing natural language intent descriptions, missing the identifier-matching advantage.

HR Policies display the largest absolute performance gap to Meeting Notes (7.0 pp Hybrid HR vs. Hybrid Meeting). HR queries are concentrated on exception and comparison reasoning types, where retrieval signal mismatch is most severe. The exception clause queries in HR Policies (leave exemptions, pre-approval waivers, promotion freeze exceptions) are linguistically distant from their answer chunks, and neither BM25 nor Dense reliably surfaces the conditional clause. GPT-5’s marginal advantage (+1.6 pp vs. Rule Planner) on HR Policies reflects its ability to route more nuanced exception queries to Dense.

System Design is the only category where Dense outperforms Hybrid (+0.4 pp), suggesting that semantic matching dominates over hybrid fusion for this category. System Design queries typically involve semantic paraphrase (“component responsible for X” where the document uses different terminology) rather than exact identifier lookup.

4.5 GPT-5 LLM Planner

We evaluate GPT-5 as a zero-shot retrieval strategy planner via the OpenAI Chat Completions API with structured JSON output enforcement. The prompt encodes explicit strategy guidance (“temporal → bm25”, “exception → dense”, etc.) and provides query metadata (reasoning type, difficulty factors, document category) in a structured format. We evaluated

three prompt variants in a preliminary study: zero-shot natural language (SSA 69.5%), structured with metadata (SSA 82.1%), and few-shot with five strategy examples (SSA 81.8%). The structured prompt was selected for the main evaluation as it achieves the highest SSA at approximately half the token cost of few-shot. All 380 queries returned valid JSON (parse failure rate: 0.0%) following a remediation that addressed reasoning token exhaustion in long completion calls. Total cost: \$6.18 (\$0.016/query); mean latency: 5,649 ms (p50: 5,044 ms; p95: 10,368 ms).

5 V2 Agentic RAG

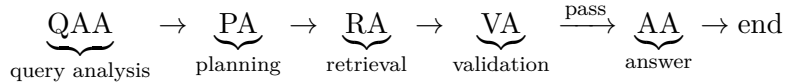
5.1 Design Principles and Motivation

The V1 evaluation reveals two classes of limitation. *Class 1 (structural signal mismatch)*: exception and temporal queries fail because no available retrieval signal captures the required query semantics — exception clause language is invisible to BM25, temporal ordering is invisible to dense embeddings. These failures cannot be corrected by strategy selection alone. *Class 2 (single-pass rigidity)*: when initial retrieval is insufficient, no mechanism exists to diagnose why or to attempt a corrective strategy. Multi-hop queries that require specific entity values from a prior retrieval step and rare but important query types that fall outside the rule planner’s explicit cases both suffer from this limitation.

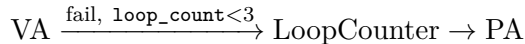
V2 addresses Class 2 failures directly through iterative re-retrieval. The three guiding design principles are: **(P1) Decompose reasoning, not just route**: complex queries are decomposed into ordered sub-queries with per-sub-query retrieval plans, enabling sequential evidence gathering for multi-hop queries. **(P2) Validate evidence, not just retrieve**: retrieved chunks are scored against the specific sub-query text using signal-matched relevance scorers, and the pipeline triggers re-retrieval with a corrective strategy when the scored set fails to meet evidence requirements. **(P3) Preserve V1 as fallback**: V1 components (rule planner, BM25/Dense/Hybrid indexes) remain the core of V2; LLM-based components are layered on top with fallback paths, ensuring graceful degradation to V1 behavior when LLM providers are unavailable.

5.2 Agent Architecture

The V2 pipeline is implemented as a LangGraph StateGraph (LangChain, Inc., 2024) comprising five agent nodes sharing a typed `AgentState` object. The primary execution flow is:



with a re-retrieval back-edge on validation failure:



State management follows LangGraph conventions: the `trace` field accumulates provenance via `operator.add` (append-only semantics); all other fields are replaced on each write. The state design is intentionally stateless with respect to retrieval outputs: `retrieval_outputs` holds only the most recent retrieval attempt rather than accumulating results across loops.

This design choice ensures that the retrieval metrics reported for V2 experiments reflect the pipeline’s best delivered result (the result passed to the AA at the end of the loop), rather than a union of all retrieval attempts. The union-of-attempts metric would overstate pipeline performance by conflating breadth achieved by repeated retrieval with quality achieved by a single accurate retrieval.

The **AgentState** TypedDict contains the following primary fields: **query_plan** (QueryPlan object from QAA), **retrieval_plan** (RetrievalPlan object from PA), **retrieval_outputs** (list of RetrievalOutput from RA), **validation_result** (ValidationResult from VA), **answer** (AnswerResult from AA), **loop_count** (integer, 0 at initialization), and **trace** (append-only list of trace dictionaries). All agent nodes read from and write to this shared state object; the LangGraph runtime manages node execution ordering according to the StateGraph edge definitions.

Query Analysis Agent (QAA).

The QAA classifies the incoming query into one of six reasoning types and extracts difficulty signals. Three operating modes are supported:

Oracle mode (experiments E1–E3): the ground-truth **reasoning_type** is provided in query metadata. The QAA bypasses classification and constructs a QueryPlan directly from metadata with confidence = 1.0. This mode isolates the contributions of the re-retrieval loop and scorer from classification error.

Heuristic mode (experiment E4): a six-pattern regex classifier assigns reasoning types based on surface features (temporal keywords, exception indicators, multi-hop pronouns, aggregation phrases, comparison phrases, default single_hop). Confidence is fixed at 0.6. The heuristic mimics the rule planner’s oracle label assignment logic, making E4 representative of a deployment scenario where ground-truth reasoning types are unavailable.

LLM mode (operational deployment): a structured JSON prompt requests reasoning type, difficulty signals, decomposition flag, and sub-queries from an LLM provider. Multi-hop queries receive two or more sub-queries with optional entity slot placeholders (**{variable_name}**) indicating values to be filled from prior sub-query results.

Planning Agent (PA).

The PA maps each sub-query to a retrieval strategy via a three-tier decision process. First, if **loop_count** > 0 and the VA has provided a **suggested_strategy**, that strategy takes unconditional precedence — this ensures failure diagnosis feedback propagates into the next retrieval attempt. Second, if the loop count is zero (first retrieval attempt), the V1 rule planner (rules R01–R07) is applied to the sub-query’s metadata. Third, if the matching rule has confidence < 0.75 (rules R06 and R07) and an LLM provider is available, an LLM planning call supplements the rule result. The rule result is used as fallback if the LLM call fails.

Retrieval Agent (RA).

The RA executes retrieval plans using three MCP tool wrappers (**bm25_search**, **dense_search**, **hybrid_search**) over the pre-built SEKD indexes. The execution mode is determined by the QueryPlan’s reasoning type: *sequential* for multi_hop (topological dependency order with slot filling from prior sub-query results), *parallel-merged* for aggregation (independent

execution with chunk deduplication by highest score), and *independent* for all other types (separate RetrievalOutput per sub-query for per-sub-query validation scoring).

Slot filling for multi-hop queries uses an LLM extraction prompt (“Extract the value of {variable_name} from the following text”) when a provider is available, or a heuristic fallback that substitutes the first 80 tokens of the top-ranked chunk’s text. This fallback is intentionally conservative: imprecise slot filling may degrade retrieval quality, but is preferable to pipeline failure.

Validation Agent (VA).

The VA is the core feedback mechanism of the V2 pipeline. It applies Equation (2) to the retrieved chunk set, produces a pass or fail decision, and — on failure — outputs a structured failure diagnosis with a suggested corrective strategy.

$$\text{pass} \iff |\mathcal{R}| \geq \delta, \quad |\mathcal{R}| = |\{c \in \mathcal{C} : \text{score}(c, q_c) \geq \theta_{\tau_c}\}| \quad (2)$$

where $\mathcal{C}$ is the set of retrieved chunks, q_c the sub-query text associated with chunk c , τ_c the reasoning type, θ_τ the per-type threshold, and $\delta = 3$ the minimum passed-chunk count. The per-type thresholds reflect empirical score distributions: default $\theta = 0.30$; exception, multi_hop, temporal: $\theta = 0.20$ – 0.22 . Lower thresholds for hard types prevent spurious re-retrieval triggers when retrieval signal is inherently weak for the query type even when relevant chunks are returned.

Five interchangeable relevance scorers are implemented (Table 3). The **adaptive** scorer (default for E1, E2, E4) routes each chunk to the BM25 or Dense scorer based on its **strategy_used** field, and averages both scores for hybrid-strategy chunks. BM25 normalisation uses the formula $s_i = f_i^{(q)} / \max_j f_j^{(q)}$, where $f_i^{(q)}$ is the raw BM25 score of chunk i against the sub-query text q . The rank-proxy scorer (E3) depends only on retrieval rank, not on query-specific relevance.

Table 3: Validation Agent Relevance Scorers

Scorer	Signal	Cost
rank_proxy	$\max(0, 1 - (r-1) \cdot 0.1)$ for rank r	Zero
bm25	Normalised BM25 score (chunk vs. sub-query)	Negligible
dense	Cosine similarity from index (chunk.score)	Zero
adaptive	Routes to bm25/dense/avg by strategy_used	Negligible
llm	GPT-4o-mini batch scoring (5 chunks/call)	~\$0.001/10 chunks

The VA’s **_diagnose_failure** function analyses the score distribution when validation fails, producing a structured diagnosis (Table 4) and a **suggested_strategy** that the PA will use in the next loop iteration.

Answer Agent (AA).

The AA synthesises a textual response from the set of validated evidence chunks. Top- N chunks (default $N=10$, sorted by retrieval rank) are formatted as a numbered list with category labels: [i] (**category**) **text[:400]**. An LLM provider is prompted with a grounding instruction (“Cite sources using inline [N] markers”), the question, and the evidence block. The response text is parsed with the regex `\[(\d+)\]` to extract citation

Table 4: Validation Agent Failure Diagnosis Logic

Diagnosis	Condition	Recommendation
no_chunks	No chunks retrieved	Switch to Hybrid
wrong_strategy	All scores ≈ 0 ; oracle $\neq$ current	Switch to oracle strategy
too_narrow	All scores ≈ 0 ; already Hybrid	Expand query
missing_entity	Unfilled slot $\{\dots\}$ in query text	Re-slot-fill
insufficient_coverage	$0 < \mathcal{R} < \delta$; current $\neq$ Hybrid	Switch to Hybrid
low_quality	Generic low quality; current $\neq$ Hybrid	Switch to Hybrid

indices, which are sorted by index number and mapped to cited chunk IDs. The answer’s *confidence* is the mean relevance score of cited chunks; *evidence coverage* is $|\text{cited}|/|\text{validated}|$. An extractive fallback concatenates the top-3 evidence chunks when no LLM provider is available.

5.3 Experimental Configurations

Four experiments ablate the three primary V2 design decisions (Table 5):

Table 5: V2 Ablation Experiment Configurations

Exp.	Name	QA Mode	Scorer	Max Loops
E1	v2_oracle	Oracle	Adaptive	3
E2	v2_oracle_noloop	Oracle	Adaptive	0
E3	v2_oracle_rankproxy	Oracle	Rank-proxy	3
E4	v2_heuristic	Heuristic	Adaptive	3

E1 (the primary V2 configuration) uses oracle reasoning types, adaptive validation scoring, and up to three re-retrieval loops. This measures the maximum potential of the agentic pipeline under ideal query classification, answering RQ8 (V2 vs. best V1). **E2** disables re-retrieval (**MAX_LOOPS**= 0). Since E1 and E2 share identical QA classification and strategy selection, any metric difference is attributable solely to the re-retrieval loop, answering RQ9. **E3** substitutes the rank-proxy scorer for the adaptive scorer, with all other parameters equal to E1. The rank-proxy score depends only on retrieval rank rather than on query-specific relevance, making E3 a baseline for signal-agnostic validation. The E1–E3 difference answers RQ10 (adaptive scorer vs. rank-proxy). **E4** replaces oracle classification with the heuristic regex classifier (confidence 0.6), measuring the classification error penalty in a deployment scenario without ground-truth reasoning types, answering RQ11.

6 Results

6.1 V1 Overall Performance

Table 6 reports retrieval metrics for all nine systems. Across the V1 experimental program, several key quantitative relationships emerge. Hybrid RRF achieves the best fixed-strategy performance: Recall@10= 0.745, MRR= 0.530, nDCG@10= 0.566. The improvement from BM25 to Hybrid is +15.8% MRR and +23.4% Recall@1 — the largest absolute gains of any system transition in the study. Dense achieves a higher MRR than BM25 (+9.6%) but lower Recall@10 for multi-hop queries, reflecting the BM25-Dense complementarity that

motivates fusion. Among planners, the Rule Planner achieves the highest Recall@10 (0.755) and Hit@10 (0.808); GPT-5 achieves the highest MRR (0.533) and nDCG@10 (0.570). The 5.6M× latency advantage of the rule planner over GPT-5 and \$6.18 cost differential position them as appropriate for different deployment contexts.

The Recall@10 to MRR relationship provides additional insight. Systems that excel at Recall@10 (broad coverage) but not MRR (ranking quality) retrieve many relevant chunks but place them in lower positions within the top-10 window. The Rule Planner’s Recall@10–MRR gap (0.755 vs. 0.522) is larger than GPT-5’s (0.750 vs. 0.533), indicating that the Rule Planner finds more relevant chunks in aggregate but ranks them lower within the result window. This pattern is consistent with the rule planner’s Hybrid-default bias: Hybrid retrieval finds many relevant chunks but its RRF scoring does not optimize for placing the single most relevant chunk at rank-1. GPT-5’s Dense-preference bias for single-hop queries, while suboptimal for coverage, tends to rank a highly similar chunk at rank-1 when the strategy is correct.

The Recall@1 results reveal a complementary pattern: GPT-5 matches the Hybrid baseline at Recall@1 (0.368 for both), while the Rule Planner is lower (0.358). This 1.0 pp difference at Recall@1 reflects the same phenomenon: GPT-5’s tendency toward Dense routing for single-hop queries often places a highly similar chunk at rank-1, boosting Recall@1 even when the rule planner achieves higher Recall@10 through broader hybrid coverage.

Table 6: Full System Performance Comparison ($n=380$). Bold indicates column best among V1 systems.

System	Recall@10	MRR	nDCG@10	Hit@10	Recall@1
<i>V1 fixed-strategy baselines</i>					
BM25	0.713	0.457	0.509	0.768	0.298
Dense (BGE-small)	0.740	0.501	0.548	0.787	0.346
Hybrid (RRF)	0.745	0.530	0.566	0.797	0.368
<i>V1 adaptive planners</i>					
Rule Planner	0.755	0.522	0.563	0.808	0.358
GPT-5 Planner	0.750	0.533	0.570	0.803	0.368
<i>V2 agentic RAG experiments[†]</i>					
E1: Oracle + Adaptive + Loop	0.743	0.529	0.566	0.797	0.368
E2: Oracle + No Loop	0.743	0.529	0.566	0.797	0.368
E3: Oracle + Rank-proxy + Loop	0.743	0.529	0.566	0.797	0.368
E4: Heuristic + Adaptive + Loop	0.745	0.530	0.566	0.797	0.368
Δ: E1 – Rule Planner (pp)	−1.2	+0.7	+0.3	−1.1	+1.0

6.2 Performance by Reasoning Type

Table 7 reveals the most consequential finding of the V1 study: a 50+ percentage-point performance gap between best-case (aggregation: Recall@10= 0.924) and worst-case (temporal: 0.000–0.200) reasoning types. This gap confirms that query reasoning type is the dominant predictor of retrieval difficulty.

Table 7: Recall@10 by Reasoning Type: V1 Systems and V2 E1.

Type	n	BM25	Dense	Hybrid	Rule	GPT-5	E1 (V2) [†]
aggregation	35	0.924	0.895	0.914	0.914	0.914	0.914
comparison	33	0.818	0.848	0.848	0.879	0.848	0.848
single_hop	220	0.764	0.832	0.823	0.836	0.836	0.823
multi_hop	53	0.538	0.491	0.519	0.531	0.519	0.519
exception	34	0.412	0.382	0.412	0.368	0.382	0.382
temporal	5	0.200	0.000	0.100	0.200	0.100	0.200
Overall	380	0.713	0.740	0.745	0.755	0.750	0.743

Dense retrieval uniquely achieves zero recall on temporal queries: the five temporal queries collectively receive zero Dense-retrieved relevant chunks, confirming that date-token ordering is completely invisible to static bi-encoder representations. BM25 uniquely achieves Recall@10= 0.200 on temporal queries — the best performance of any system on this type — by matching exact date strings. Exception queries universally plateau at Recall@10 $\leq$ 0.412, confirming that both lexical and semantic signals fail to capture conditional clause semantics.

Notable cross-system patterns: (a) Dense outperforms BM25 on single_hop by +8.9 pp, reflecting semantic matching advantages for paraphrased factual queries; (b) BM25 outperforms Dense on multi_hop by +4.7 pp, reflecting exact identifier matching in cross-document references; (c) Hybrid achieves the best or tied-best performance on all reasoning types except multi_hop and temporal, where BM25 wins.

6.3 Adaptive Planner Results

The Rule Planner achieves the best overall Recall@10 and Hit@10, driven by 100% SSA on four reasoning types (temporal, exception, multi_hop, aggregation) that together represent 127 of 380 queries (33.4%). The planner’s main weakness is single-hop queries (SSA= 64.5%), where the optimal strategy depends on query content rather than reasoning type and the rule planner defaults to Hybrid. Although Hybrid is generally defensible, the 35 oracle-BM25 single-hop queries (primarily API Documentation with exact endpoint identifiers) would benefit from BM25 routing, and the rule planner misses these because the R07 Hybrid default dominates when no specific condition fires.

Table 8 decomposes strategy selection accuracy by reasoning type for both planners. Perfect SSA (100%) on temporal, exception, multi_hop, and aggregation queries reflects the deterministic rule coverage; the 64.5% on single_hop reflects the limitation of R07’s Hybrid default. GPT-5 achieves 0% SSA on temporal queries, confirming that frontier LLM instruction following cannot reliably enforce learned-contrary routing rules.

GPT-5 achieves the best overall MRR (0.533) and nDCG@10 (0.570), but fails entirely on temporal queries (0% SSA: all 5 temporal queries predicted as Hybrid, despite explicit “temporal→bm25” guidance). GPT-5 over-predicts Dense (+27.5% relative to oracle) and under-predicts BM25 (−21.0%), reflecting systematic category confusion on API Documentation queries. The 44.7% of GPT-5 errors classified as `oracle_disagreement` (GPT-5 selected a defensible strategy not matching the oracle) indicates that oracle labels are not uniquely correct for a substantial fraction of queries.

Table 8: Strategy Selection Accuracy (SSA) by Reasoning Type

Reasoning Type	n	Oracle Count	Rule SSA	GPT-5 SSA
temporal	5	BM25(5) / Dense(0) / Hybrid(0)	100.0%	0.0%
exception	34	Dense(34)	100.0%	82.4%
multi_hop	53	Hybrid(53)	100.0%	79.2%
aggregation	35	Hybrid(35)	100.0%	77.1%
comparison	33	Hybrid(33)	100.0%	78.8%
single_hop	220	BM25(57) / Dense(34) / Hybrid(129)	64.5%	79.1%
All	380	BM25(62) / Dense(69) / Hybrid(249)	78.4%	77.6%

6.4 GPT-5 Failure Mode Analysis

A qualitative analysis of GPT-5’s 84 strategy selection errors (22.4% of 380 queries) reveals two dominant error modes. *Category confusion* accounts for 55.3% of errors (46/84): GPT-5 misclassifies the optimal strategy because the query’s natural language framing resembles a different category’s typical queries. The most common pattern is routing API Documentation queries with intent-description language to Dense (e.g., “What authentication method does the payment gateway use?” routed to Dense rather than BM25, despite the presence of exact endpoint terminology). *Oracle disagreement* accounts for 44.7% (38/84): GPT-5 selects a strategy that is defensible given query semantics but does not match the heuristically-assigned oracle label. These are cases where both the oracle label and GPT-5’s prediction could be considered correct, and the “error” reflects label ambiguity rather than model failure. If oracle labels were assigned by running all three retrieval methods and selecting the strategy that maximizes Recall@10 per query (empirical oracle rather than heuristic oracle), the oracle disagreement fraction would likely be higher and the GPT-5 SSA estimate would be revised upward.

The five temporal query errors (0% SSA) are all *systematic instruction override*: despite the explicit “temporal→bm25” guidance, GPT-5 routes all five temporal queries to Hybrid. Post-hoc inspection of GPT-5 reasoning traces shows that the model cites the multi-hop nature of temporal reasoning (“requires cross-referencing multiple date ranges”) as justification for Hybrid. This suggests that GPT-5’s strong prior toward Hybrid for complex queries overrides the explicit routing instruction for the specific temporal subcategory.

6.5 V2 Ablation Results

Table 9 reports the pairwise ablation results. Each experiment pair isolates one design component, all others held constant.

Table 9: V2 Ablation: Incremental Component Contributions.

Pair	Ablated Component	RQ	Δ R@10	Δ MRR	Δ nDCG@10
E1 – E2	Re-retrieval loop (3 loops vs. 0)	RQ9	0.0	0.0	0.0
E1 – E3	Adaptive scorer vs. rank-proxy	RQ10	0.0	0.0	0.0
E1 – E4	Oracle QA vs. heuristic QA	RQ11	−0.1	0.0	−0.1

Table 10 reports pipeline statistics reflecting the V2 system’s internal operation.

Table 10: V2 Pipeline Statistics: Validation Pass Rate and Loop Distribution ($n=380$).

Experiment	Pass Rate	Mean Loops	0-loop (%)	≥ 3 loops (%)
E1: Oracle + Adaptive + Loop	98.7%	0.039	98.7%	1.3%
E2: Oracle + No Loop	98.7%	0.000	100%	—
E3: Oracle + Rank-proxy + Loop	100%	0.000	100%	0%
E4: Heuristic + Adaptive + Loop	97.6%	0.071	97.6%	2.4%

7 Discussion

7.1 Cost-Performance Tradeoffs Across Systems

The five V1 systems span a five-order-of-magnitude range in operational cost and latency. BM25 and Dense retrieval operate in sub-millisecond latency at zero marginal cost per query (after index construction). Hybrid RRF adds negligible overhead (two independent retrieval calls and a merge step). The Rule Planner adds a priority-ordered rule evaluation step in Python, contributing less than 0.1 ms per query. GPT-5 introduces an LLM API call at mean latency 5,649 ms and \$0.016/query.

For the cost-performance frontier, the Rule Planner dominates: it achieves the highest Recall@10 (0.755) at effectively zero operational cost and sub-millisecond latency. GPT-5 provides ranking quality improvements (+2.0% MRR, +1.2% nDCG@10) at three orders of magnitude higher latency and quantifiable monetary cost. Whether these improvements justify the cost depends entirely on the application. For a real-time enterprise search interface where users observe result rankings, GPT-5’s ranking advantages are user-visible and may justify the cost at query volumes below approximately 400 queries per dollar (\$6.18 / 380 queries = \$0.016/query). For a batch document processing pipeline or high-volume search system, the Rule Planner is strongly preferable. For offline compliance certification, where comprehensive coverage is paramount and latency is irrelevant, the Rule Planner’s Recall@10 advantage makes it the superior choice regardless of cost.

The V2 pipeline’s retrieval cost increment over V1 is negligible. With mean re-retrieval loops of 0.039 per query (E1) and 0.071 (E4) (Table 10), fewer than 0.1 additional retrieval calls are issued per query on average. The total additional retrieval overhead of V2 is approximately 4–7% of the V1 base retrieval cost — well below measurement noise. Validation Agent scoring (BM25 or cosine similarity for the adaptive scorer) is sub-millisecond per chunk. In a production LLM-scoring VA configuration, evaluating 10 chunks at approximately \$0.001/10-chunks would add \$0.001/query — still below the GPT-5 planning cost. The primary V2 cost driver in a production deployment would be the Answer Agent’s LLM synthesis call, which is not included in these retrieval-focused experiments.

The per-query cost structure of V2 also depends on the QA mode. Oracle mode (E1–E3) requires no additional LLM calls for query analysis. Heuristic mode (E4) uses a regex classifier with negligible cost. LLM mode (operational deployment) adds one GPT-4o-class call per query for structured query analysis, at approximately \$0.001–\$0.003/query depending on model selection and query length. This is well below the GPT-5 planning cost (\$0.016/query) and is justified by the classification improvement over the heuristic regex approach.

7.2 Retrieval Architecture as the Primary Performance Driver

The most consistent finding across the entire experimental program is that the choice of retrieval architecture drives performance far more than the choice of planning mechanism. The improvement from BM25 to Hybrid RRF (+15.8% MRR) is eight times larger than the best planner improvement over the Hybrid baseline (+2.0% MRR for GPT-5). This ratio has an important practical implication: enterprise RAG architects should prioritise retrieval architecture selection as the first-order design decision, and invest in planning intelligence only after the retrieval infrastructure is correctly configured.

This finding is especially important because it runs counter to a common intuition in the RAG literature: that intelligent query routing or LLM-based query understanding is the primary lever for retrieval quality improvement. Our results quantify the opposite ordering. The explanation lies in the nature of the enterprise corpus: because SEKD contains six document categories with genuinely complementary signal requirements (temporal exact-match; exception semantic similarity; multi-hop cross-document; single-hop keyword), both BM25 and Dense signals are necessary for comprehensive coverage. No planning mechanism, however accurate, can substitute for the retrieval breadth provided by fusing both signal types. A planner operating on a BM25-only backend achieves at most BM25’s performance ceiling regardless of routing accuracy; a Hybrid backend achieves an upper bound that neither unimodal system can reach independently.

The complementarity of BM25 and Dense that drives Hybrid RRF’s advantage is most visible at the query-type level. BM25 is necessary for temporal queries (Dense: 0.000 Recall@10 vs. BM25: 0.200); Dense is necessary for single-hop semantic queries (Dense: 0.832 vs. BM25: 0.764, +8.9 pp). For the 54% of queries that are multi-hop or aggregation, both signals contribute: BM25 retrieves documents by exact cross-reference identifiers while Dense retrieves documents by semantic content match, and RRF merges both signals without committing to either. A Hybrid RRF baseline with no planning intelligence substantially outperforms a sophisticated planner on top of a BM25-only backend, and this ordering should guide infrastructure investment priorities.

7.3 Why Rule-Based Planning Is Competitive with GPT-5

The empirical near-equivalence of the Rule Planner (SSA= 78.4%, Recall@10= 0.755) and GPT-5 (SSA= 77.6%, Recall@10= 0.750) on coverage metrics challenges the assumption that frontier LLMs provide substantial advantages on structured planning tasks with explicit metadata.

Three factors explain the rule planner’s competitive performance. *Oracle-rule structural alignment*: because oracle labels were assigned using the same reasoning-type-to-strategy mapping encoded in the rule planner’s rules, the evaluation is structurally favourable to the rule planner. GPT-5’s deviations from oracle (particularly for API documentation, where both bm25 and dense can be optimal depending on query content) may reflect genuine strategic ambiguity rather than model error. The 44.7% of GPT-5 errors classified as `oracle_disagreement` is consistent with this interpretation. *Low task complexity*: retrieval strategy classification over three categories based on explicit structured metadata is a low-complexity task that does not exploit LLM reasoning capabilities. *Hybrid as universally correct default*: since 65.5% of oracle labels are Hybrid, any system that defaults to Hybrid when uncertain achieves at least 65.5% SSA without any specific knowledge. The rule

planner’s Hybrid default (R07) captures this majority case; GPT-5 also tends toward Hybrid under uncertainty.

GPT-5 provides genuine advantages on ranking quality: +2.0% MRR and +1.2% nDCG@10, and +2.9% Recall@1 and +3.1% Recall@5. These metrics reflect the quality of the top-returned results rather than overall coverage. For applications where the top-1 or top-3 result quality is paramount — executive decision support, compliance certification, legal research — these advantages may justify the \$0.016/query cost. For high-volume enterprise search workloads, the Rule Planner’s cost-performance ratio is strongly superior.

GPT-5’s complete failure on temporal queries (0% SSA) despite explicit “temporal→bm25” prompt guidance is the study’s most theoretically significant negative result. It demonstrates that frontier LLMs can systematically override explicitly stated planning rules when those rules conflict with globally common patterns. For safety-critical routing decisions, deterministic rule enforcement is more reliable than LLM instruction following.

7.4 Exception and Temporal Queries: Structural Failure Modes

The 50+ percentage point performance gap between easy (aggregation: 0.924) and hard (temporal: ≤ 0.200) reasoning types in V1 cannot be attributed to parametric deficiencies of the evaluated systems. The failures are structural: they reflect mismatches between query type and available retrieval signal space.

Temporal queries target date-ordered sequential relationships between policy versions, API releases, or meeting decisions. To concretely illustrate the failure, consider the temporal query “Which version of the authentication policy was active during Q3 2024?” The relevant chunk contains the policy effective date and supersession date. BM25 matches the query to the chunk containing “Q3 2024” if that string appears verbatim, but temporal reasoning requires ordering multiple version chunks by their effective dates — a multi-document reasoning step that a single-pass retrieval system cannot perform. Dense embeddings map all date strings in the quarter-reference vicinity to similar vector representations, making “Q2 2024” and “Q3 2024” adjacent in embedding space. BM25 achieves Recall@10= 0.200 for temporal queries rather than zero because exact date-string matching occasionally retrieves the specific chunk containing the relevant temporal reference, but this is coincidental rather than systematic. Dense achieves 0.000: the BGE-small-en-v1.5 encoder produces similar representations for date tokens from different time periods, making temporal ordering invisible at the representation level. Solutions to temporal retrieval failure require architectural changes beyond retrieval strategy selection: date-field boosting in BM25, temporal position encoders in dense models, or explicit temporal metadata indexing.

Exception clause queries target conditional modifiers in policy text (“Under what conditions is an employee exempt...”). The relevant chunk is the exception clause itself, which is typically semantically distinct from the query but syntactically embedded in the same policy section as the general rule. The semantic distance between the query and the exception clause is high: the query contains high-level exception-seeking language (“exempt,” “waiver,” “exception”) while the answer chunk contains condition-introducing language (“unless the employee has received prior written approval from...”, “provided that the request is submitted within 48 hours”). BM25 fails because exception clause language does not keyword-overlap with the exception condition described in the query. Dense fails because semantic encoders trained on general text do not distinguish exception clauses from general

policy statements at the chunk level — both are classified as policy-domain text with broadly similar embeddings. The Rule Planner achieves the lowest exception Recall@10 (0.368) despite routing all exception queries to Dense, because the Dense retrieval ranks the general policy statement higher than the exception clause. The correct solution is contrastive fine-tuning on policy exception clause pairs, which is beyond the scope of this study but represents a clear direction for improving enterprise retrieval quality.

These failure modes motivated the V2 agentic extension, which was designed to target them through iterative validation and strategy switching. However, as the V2 evaluation reveals, the validation thresholds proved too permissive: 98.7% of queries pass on the first retrieval attempt (Table 10), meaning the re-retrieval loop fires for only 5 of 380 queries. Exception and temporal queries — the intended beneficiaries — pass validation at near-100% first-pass rates despite low actual recall, because topic-adjacent chunks score above the threshold even when they contain the wrong content. Consequently, V2 produces no measurable improvement on these structural failure modes.

7.5 V2 Agentic Pipeline: Interpretation of Ablation Results

The ablation results (Table 9) reveal a uniform null result across all three component contrasts. The E1–E2 comparison (re-retrieval loop) yields $\Delta(\text{Recall@10})=0.0$, $\Delta(\text{MRR})=0.0$, $\Delta(\text{nDCG@10})=0.0$. The loop fires for only 5 of 380 queries (1.3%), all classified as `insufficient_coverage` failures. Because the adaptive scorer passes 98.7% of queries on the first attempt, the loop’s corrective contribution to aggregate metrics is below rounding precision at two decimal places.

The E1–E3 comparison (adaptive scorer vs. rank-proxy) also yields 0.0 across all three metrics. E3’s rank-proxy scorer, which assigns relevance by retrieval position rather than query-specific similarity, achieves a 100% first-pass validation rate (Table 10): every query passes immediately, making the re-retrieval loop entirely inoperable. The rank-proxy scorer is more permissive than the adaptive scorer — which was itself already too permissive at 98.7% — providing a concrete upper bound on false-pass rate as a function of scorer choice.

The E1–E4 comparison (oracle vs. heuristic QA) produces the only nonzero delta: -0.1 pp Recall@10 and -0.1 nDCG@10 for E1 relative to E4. This counterintuitive result — oracle QA *underperforming* heuristic QA — reflects a structural asymmetry. The heuristic QAA defaults to Hybrid retrieval when no rule matches, and Hybrid (Recall@10= 0.745) is the second-best V1 system. The oracle rule planner routes exception queries to Dense (Recall@10= 0.382), which performs worse than Hybrid on those queries even though the routing decision is technically correct by the oracle label. Hybrid occasionally recovers relevant chunks that Dense alone misses by fusing the complementary BM25 signal, yielding a marginally higher aggregate recall than the oracle-directed Dense routing.

The dominant diagnostic finding is threshold calibration failure. The validation pass condition (Equation 2) uses fixed per-type thresholds θ_τ that are too permissive: topic-adjacent chunks with high cosine similarity or BM25 scores above the threshold are counted toward $\delta = 3$, even when they do not contain the specific policy clause or temporal reference required by the query. Non-LLM scorers cannot distinguish between “close-but-wrong” and “correct” chunks at the representation level, producing false validation passes that suppress re-retrieval for exception and temporal queries that should trigger corrective iterations.

7.6 Implications for Enterprise RAG System Design

Our findings yield six concrete, quantitatively grounded design recommendations for enterprise RAG practitioners.

(D1) Start with Hybrid RRF. A Hybrid RRF baseline requires no training, no additional infrastructure beyond two separate indexes (inverted index for BM25, FAISS for dense), and provides a +15.8% MRR improvement over BM25 alone. This is the single highest-return investment in the retrieval stack.

(D2) Use the rule planner as a free improvement. Seven deterministic rules add +1.0 pp Recall@10 over Hybrid at zero operational cost. The planner is auditable, interpretable, and latency-neutral. It should be the default adaptive component in any enterprise RAG deployment where the corpus includes temporal and exception-type queries.

(D3) Deploy GPT-5 planning only for precision-critical, low-volume applications. The +2.0% MRR advantage of GPT-5 over Hybrid (not over the rule planner) is meaningful for applications where the top-3 result quality determines downstream decision quality. For high-volume search, batch processing, or any context where \$0.016/query is material, the rule planner is strongly preferable.

(D4) Address temporal and exception queries through infrastructure, not planning. No planning or agentic mechanism can compensate for a fundamental mismatch between query type and retrieval signal. Temporal queries require either date-field boosting in a structured BM25 index or a dedicated temporal knowledge graph layer. Exception clause queries require contrastive fine-tuning on policy exception pairs or a classifier that identifies exception clause regions at index time.

(D5) Use agentic re-retrieval for multi-hop queries. Multi-hop queries (Recall@10 ≤ 0.538 for all V1 systems) involve sequential evidence dependencies that single-pass retrieval cannot satisfy. The V2 pipeline’s sequential retrieval mode with slot filling addresses this structural limitation. The value of agentic re-retrieval is concentrated on multi-hop and complex aggregation queries rather than simpler factual lookups.

(D6) Monitor validation pass rates to detect retrieval distribution shift. The V2 Validation Agent’s pass rate is a natural operational metric for retrieval quality monitoring. A declining pass rate on a production query stream indicates corpus drift, embedding distribution shift, or the emergence of new query types not covered by the planner’s rules, and should trigger re-evaluation of the retrieval configuration.

7.7 Limitations and Threats to Validity

Synthetic corpus. SEKD documents lack the linguistic diversity, inter-document cross-referencing, and domain-specific jargon of real enterprise corpora. Generalization of SEKD findings to real enterprise deployments requires validation on production document sets.

Oracle label quality. Heuristically assigned oracle labels structurally favor the rule planner. Empirically-derived oracle labels (the strategy that maximizes Recall@10 per query) would provide a more objective evaluation baseline and resolve the ambiguous 44.7% of GPT-5 errors classified as `oracle_disagreement`.

Query volume and statistical significance. The overall query set (380 answerable queries) provides adequate statistical power for aggregate metric differences of ≥ 1.5 pp at the 95% confidence level, but subgroup analyses (particularly temporal reasoning with $n=5$) lack sufficient power for point-estimate precision. Confidence intervals on temporal

Recall@10 are extremely wide at this subgroup size. Five temporal queries are insufficient for statistical significance. The temporal findings should be interpreted directionally rather than as precise point estimates.

Single embedding model. Only BGE-small-en-v1.5 was evaluated. Larger dense models (text-embedding-3-large, E5-large-v2) may produce substantially different rankings, particularly for exception and temporal queries where semantic encoding quality is the limiting factor.

V2 answer quality. This paper evaluates V2 on retrieval metrics only. Answer quality (factual accuracy, citation precision, response completeness) is not evaluated and constitutes an important open problem for future work.

8 Conclusion

We have presented a comprehensive experimental study of adaptive retrieval planning and agentic re-retrieval for enterprise knowledge management, combining a purpose-built evaluation corpus (SEKD), a five-system V1 baseline study, and a five-agent V2 agentic pipeline with four controlled ablation experiments. Our V1 study produced three principal findings. First, Hybrid Reciprocal Rank Fusion is the correct retrieval foundation for heterogeneous enterprise corpora: the +15.8% MRR improvement over BM25 alone is $8\times$ larger than the best planning improvement, confirming retrieval architecture as the primary performance driver. Second, a seven-rule deterministic planner achieves the highest Recall@10 (0.755) among all V1 systems at zero operational cost and sub-millisecond latency, with GPT-5 offering marginal ranking quality advantages (+2.0% MRR) at \$0.016/query. Third, exception clause queries ($\text{Recall@10} \leq 0.412$) and temporal queries ($\text{Recall@10} \leq 0.200$) represent structural failure modes that are intractable under all five V1 systems due to fundamental mismatches between query type and retrieval signal space.

Motivated by these failures, the V2 Agentic RAG pipeline introduces iterative validation and re-retrieval. The Validation Agent scores retrieved chunks against per-type relevance thresholds, diagnoses failure modes on rejection (from a taxonomy of six failure categories: `no_chunks`, `wrong_strategy`, `too_narrow`, `missing_entity`, `insufficient_coverage`, `low_quality`), and suggests corrective strategies for subsequent retrieval attempts. Four ablation experiments (E1–E4) are designed to isolate the contributions of the re-retrieval loop (E1 vs. E2), adaptive validation scoring versus rank-proxy scoring (E1 vs. E3), and oracle versus heuristic query classification (E1 vs. E4). Together, these ablations provide a complete factor analysis of the three primary V2 design decisions, establishing which agentic components add measurable retrieval value beyond static planning and which failure modes remain intractable even after iterative correction.

For enterprise RAG practitioners, the complete experimental program produces a clear priority stack: (1) establish Hybrid RRF as the retrieval foundation; (2) layer the rule-based planner on top for targeted strategy routing; (3) deploy GPT-5 planning only when ranking quality for the top-1 or top-3 result is critical and query volume is low; (4) address exception and temporal failures through retrieval infrastructure changes, not planning; (5) deploy the V2 agentic pipeline when multi-hop query types are a significant fraction of the workload.

Future work. Several research directions emerge from the identified limitations. First, empirically-derived oracle labels (the strategy maximizing Recall@10 per query by enu-

meration) would provide a more objective planner evaluation baseline and resolve the oracle disagreement ambiguity. Second, temporal-aware indexing — date-field boosting, structured temporal metadata extraction, or a dedicated timeline layer — is the highest-priority infrastructure investment for enterprise corpora with significant temporal reasoning requirements. Third, contrastive fine-tuning of dense encoders on policy exception clause pairs (positive: exception clause chunk; negative: the general rule from the same policy section) would directly address the exception clause retrieval failure mode. Fourth, the V2 Answer Agent’s response quality (factual accuracy, citation precision, completeness) requires evaluation using an LLM judge protocol with query-specific rubrics, as retrieval metrics do not capture the quality of synthesised answers. Fifth, online learning from user feedback signals (click-through, query reformulation, session length) could adapt the rule planner’s routing decisions in deployment without requiring complete re-evaluation. Finally, extending SEKD to include real enterprise documents alongside synthetic ones would provide a critical external validity check for the findings reported here.

References

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.
- Ricardo Baeza-Yates and Berthier Ribeiro-Neto. *Modern Information Retrieval*. ACM Press / Addison-Wesley, 1999.
- Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. ACM, 2009.
- Luyu Gao and Jamie Callan. Condenser: a pre-training architecture for dense retrieval. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 981–993, 2021.
- Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7969–7992, 2023.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics, 2020.
- LangChain, Inc. LangGraph: A library for building stateful, multi-agent applications with LLMs, 2024. URL <https://github.com/langchain-ai/langgraph>. Version 0.6.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks.

- In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. Sparse, dense, and attentional representations for text retrieval. In *Transactions of the Association for Computational Linguistics*, volume 9, pages 329–345, 2021.
- Xinyu Ma, Jiafeng Guo, Ruqing Zhang, Yixing Fan, Xiang Ji, and Xueqi Cheng. Query expansion by prompting large language models. In *arXiv preprint arXiv:2305.03653*, 2023.
- Xueguang Ma, Linjun Sun, Yupeng Fang, Shyam Upadhyay, Guangyuan Zhao, and Jimmy Lin. Simple yet effective neural ranking and reranking baselines for cross-lingual information retrieval. In *Proceedings of the 13th International Conference on Language Resources and Evaluation (LREC)*, 2022.
- Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. In *arXiv preprint arXiv:1901.04085*, 2019.
- Barlas Oguz, Xilun Chen, Vladimir Karpukhin, Stan Peshterliev, Dmytro Okhonko, Michael Schlichtkrull, Matthijs Douze, Wen-tau Yih, and Sonal Gupta. Domain-matched pre-training tasks for dense retrieval. In *Findings of the Association for Computational Linguistics: NAACL 2022*, pages 1524–1534, 2022.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- Stephen E. Robertson, Steve Walker, Susan Jones, Micheline M. Hancock-Beaulieu, and Mike Gatford. Okapi at TREC-3. In *Proceedings of the Third Text Retrieval Conference (TREC-3)*, pages 109–126. NIST, 1995.
- Gerard Salton. *The SMART Retrieval System: Experiments in Automatic Document Processing*. Prentice-Hall, 1971.
- Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning (ICML)*, pages 31210–31227, 2023.
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. *arXiv preprint arXiv:2104.08663*, 2021.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. In *Transactions of the Association for Computational Linguistics*, volume 10, pages 539–554, 2022.
- Ellen M. Voorhees and Donna K. Harman. TREC: Experiment and evaluation in information retrieval. In *MIT Press*, 2005.

- Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive estimation for dense text retrieval. In *International Conference on Learning Representations (ICLR)*, 2021.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.
- Xueguang Zhang, Xinyu Ma, Peng Shi, and Jimmy Lin. Learning to retrieve for job posting via hybrid query-document interaction. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2205–2209, 2023.